

Lucas Kenzo Izumi
Lucas Cordeiro Gatti
Caio Brun de Oliveira

Comparative analysis of data structures to implement sparse polynomials

Data Structures - SCC0223
University of São Paulo
Institute of Mathematics and Computer Science - ICMC
September 15th, 2026

1 SOLUTION OF THE PROJECT

For our solution, we have chosen to use a Dynamic ordered singly linked list double linked list as the data structure.

ADD first searches where the term should be by

```
1 boolean ADD(POLINOMIO *p, long long c, int g){
2     if (p != NULL && c != 0){
3         TERMO *aux = p->inicio;
4         TERMO *aux2 = NULL;
5         while (aux != NULL && aux->g > g){
6             aux2 = aux;
7             aux = aux->proximo;
8         }
9         ...
10 }
```

Listing 1 – ADD operation

p is the given polynomial, g is the given degree, and c is the given coefficient.

After finding it, it divides into 3 cases: aux is different than NULL; aux is at the start of the polynomial; any other option. The first case exists because of segmentation fault, but inside it, it checks if there's a term with the same degree as g. If not, it dynamically allocates a new term, and links this new term with aux and the next term of aux. Finally, it sets the coefficient and degree of the new term to the given coefficient and degree.

Also, putting "aux != NULL" helped with segmentation fault problems

The operations DEF, SOMA, PROD are almost entirely based on ADD. DEF A k uses ADD operation k times. SOMA, with polynomials A, B and R, is divided into 3 cases: R is equal to A; R is equal to B; R is different from A and B. In the first (or the second) case, all the terms of B (A) are added to A (B) with ADD. In the last case, all terms of A and B are added to R.

ESCALA simply goes from the beginning to the end of the list multiplying all the term's coefficient by the coefficient given. To go from the beginning to the end of the list, we used

```
1 boolean ESCALA(POLINOMIO *p, long long c){
2     TERMO *aux = p->inicio;
3     //until aux reaches the last term
4     while(aux != NULL){
5         //goes to the next term
6         aux = aux->proximo;
7     }
8     ...
9 }
```

Listing 2 – ESCALA operation

COEF searches the same ADD does. Because we chose an ordered list, we only have to search until the degree of the auxiliary pointer is less than g . Inside the while, it checks if `"aux->g == g"`, returning `aux->c` if it is. Otherwise, goes to the next term.

REMOVE uses the same logic that COEF used to search for a term with the only difference being that it uses 2 auxiliary pointer, one to search the term that will be removed, and the other that will be previous term. If it finds it, it divides into 2 cases: first term; any other term. In both cases, it links the previous term with the next term of aux. After that, aux is freed.

REMOVEMENOR goes to the last term of the polynomial with a second auxiliary pointer (aux2) that will be the previous term of aux. Then, it also divides into 2 cases: first term; any other term; and make the previous term point to NULL. To determine if its the first case, it checks if `"aux2 == NULL"`, because when its defined, its set to NULL.

Because its an ordered list, the greatest degree is in the first term. The operation returns `"p->inicio->g"`, p being the point that points to the polynomial.

IMPRIME goes from the start to the end of the polynomial, printing the coefficient and degree in the form of $c * x^g$. Because each term is separated by space, the last term is printed out of the while. If the polynomial has no terms (`p->inicio == NULL`), it prints 0.

IMPRIMEINV goes to the end of the polynomial, with 2 auxiliary pointers (aux and aux2), prints the coefficients and degrees of both. Then, until aux2 is at the start of the polynomial, it sets aux to the start, make it go to the previous term of aux2, prints its coefficients and degree, and make aux2 equal to aux, so it goes back.

LIBERA is based on REMOVE, removing the first term of the polynomial until the polynomial has no terms (`p->inicio == NULL`). It frees the name that was dynamically allocated in `"main.c"`, and p that was dinamically allocated in `"lista.c"`.

FIM is based on LIBERA and its located at `"lista.c"`. `"lista.c"` used a static sequential list to implement the polynomials that were created. It has the variable `"final"`, and when it is 0, it means that there are no polynomials in the list. The operation calls LIBERA until `"final"` is equal to 0.

2 JUSTIFICATION

Looking at the table, its hard to see why Dynamic Ordered Singly linked list is our option. So a more detailed comparison is needed.

We didn't choose DOS (or a sequential list) even though its $O(n)$ in all the operations mentioned in the table, because in ADD/REMOVE operation it has $O(n)$ in all cases,

Tabela 1 – Order of Complexity

Operation	DOSL	DODL	DOS	SUS
ADD	$O(n)$	$O(n)$	$O(n)$	$O(1)$
REMOVE	$O(n)$	$O(n)$	$O(n)$	$O(n)$
IMPRIME	$O(n)$	$O(n)$	$O(n)$	$O(n^3)$
IMPRIMEINV	$O(n^2)$	$O(n)$	$O(n)$	$O(n^3)$

DOSL: Dynamic Ordered Singly Linked List; DODL: Dynamic Ordered Double Linked List; DOS: Dynamic Ordered Sequential list; SUS: Static Unordered Sequential list

while DOSL and DODL have $O(n)$ only in the worst case. So, probably in the average case DOSL and DODL would be better. One of the advantages of DOS is going to the n -th element of a list with complexity of $O(1)$, because its pointers are sequential, but this is only used in IMPRIMEINV (which would need n operation, while DODL $2n$ (start to finish, and then finish to start)). Using a linked list has advantages in DEF, SOMA, PROD, ADD, REMOVE and LIBERA, because there's no need to shift the terms, and those operations, in general, are more used than IMPRIMEINV.

While SUS (or unordered list) has $O(1)$ in ADD, the complexity grows a lot more in IMPRIME/IMPRIMEINV than the other three data structures. With how important IMPRIME is (because its how the user can see the polynomial), its a big disadvantage to have an operation with such high complexity. Also, through the specifications, a polynomial can have up to 299.998 terms. This means a huge disadvantage for SUS (or Static lists) if the execution creates a polynomial with only 5 terms, for example. Even though dynamic allocation costs more than static allocation, it shouldn't affect more than using a low percentage of the entire list. Furthermore, using so little of the entire list creates the same problem that sparse polynomials have with creating a lot of terms with coefficient zero.

About choosing between doubly or singly linked lists, an advantage of doubly linked lists is that IMPRIMEINV has a order of complexity of $O(n)$, while singly linked list has $O(n^2)$. But, that's its only advantage. Doubly linked lists needs more pointer attributions in ADD, REMOVE and REMOVEMENOR, and also needs more memory to store double the pointers. IMPRIME may be one of the most important operations, and thats why IMPRIMEINV may not be used as much as IMPRIME. That's why we chose a doubly linked lists instead of a singly linked list.

3 MEASUREMENTS

In order to analyze how the program, and the data structures used for the implementation of the program, behave as the values grow, we are going to measure the program on the extremes of the range of possible values

First let's analyze the program's behavior given the simplest possible entries and the simplest possible function: 3 zero-degree polynomials, and the function SOMA

Unit	Value
Seconds	47.4953494
Milliseconds	47495.3494
Ticks	474953494
Minutes	0.791589156666667
Hours	0.0131931526111111
Days	0.00054971469212963

Tabela 2 – Measured execution time

Now, let's test it with the largest possible entry for each polynomial, using the same function, and compare it with the previous results

60.79/10.000

This is the execution time in ms. As it is clear, the time greatly increases. Which allows us to conclude that the largest entry took more time to be processed than the small on. Entries in between the largest and the smallest tend to generate medium amounts of time consumption per execution.

4 WEAK POINTS

- REMOVEMENOR: Without a pointer back to the previous element (like in a doubly linked list), locating the new end of the list requires traversing it from the beginning, resulting in an $O(n)$ operation regardless of where in the list the removal occurs. This is particularly costly for polynomials with many terms when REMOVEMENOR is called repeatedly, for instance, emptying a polynomial with n terms one REMOVEMENOR call at a time costs $O(n)$ per call, degrading to $O(n^2)$ in total.
- IMPRIMEINV: Since the list only links forward, printing in ascending order of degree cannot simply follow the existing pointers backward, as a doubly linked list would. Without an auxiliary structure, the only way to reconstruct the reverse order is to repeatedly traverse the list from the beginning, each time locating the next smallest degree that has not yet been printed. This results in $O(n)$ work per term, or $O(n^2)$ overall for a polynomial with n terms, a cost that grows quite a lot for large polynomials.
- ADD: Since the polynomial is stored in an ordered singly linked list, adding a new term requires traversing the list to find the correct position according to its degree. The traversal may also be necessary to determine whether a term with the same degree already exists, in which case the coefficients must be combined. So, even

though inserting a node after its position has been found is an $O(1)$ operation, finding this position takes $O(n)$ in the worst case. This also affects operations which rely on ADD.

- **PROD:** PROD is a particularly costly operation. Firstly, if we multiply a polynomial A with n terms by a polynomial B with m terms, this operation already requires $O(nm)$ term multiplications, since every term of A must be multiplied by every term of B. Secondly, in our implementation, each resulting term is added to the result polynomial using ADD. Since our polynomial is represented by an ordered singly linked list, ADD may require traversing the list to find the correct position or a term with the same degree. Therefore, the cost of PROD can be greater than $O(nm)$, depending on the number of terms in the resulting polynomial.

5 TEST CASES

PROD/SOMA TEST CASES

PROD/SOMA A B R (R exists and has at least one term)

PROD/SOMA A B A

PROD/SOMA A B B

PROD/SOMA A A A

OBS: This tests if R/A/B are being correctly overwritten

PROD/SOMA A I R (I doesnt exist)

PROD/SOMA I A R (I doesnt exist)

PROD/SOMA U I R (U and I doesnt exist)

OBS: this tests if something happens when there's an attempt to multiply a polynomial by an inexistent polynomial (or multiply an inexistent polynomial by an inexistent polynomial)

ADD TEST CASES

ADD A 2 3 (A doesnt exist)

OBS: this tests if something happens ADD is used with an inexistent polynomial

ADD A 0 3

OBS: this tests with a term with coefficient zero will be added

ADD A 3 4 ADD A 4 4

OBS: this tests if adding a term with a degree that already exists in the polynomial will add up the coefficients of the terms

DEF A 1 3 4 ADD A -3 4

OBS: this tests if adding a term, with coefficient that's the opposite of the coefficient of a term of the polynomial, will delete it

MEMORY LEAK TEST CASES

Use it with valgrind.

DEF A 2 3 7 4 8 LIBERA A FIM

DEF A 0 LIBERA A DEF A 2 4 8 5 1 LIBERA A FIM

OBS: this tests if LIBERA operation is working correctly. The second tests if using LIBERA and DEF with some name, then using LIBERA again with same name would cause any problems. This ensures that the polynomial associated with the name will be completely deleted.

DEF A 0 REMOVEMENOR A FIM

DEF A 1 3 4 REMOVEMENOR A FIM

DEF A 2 3 4 8 1 REMOVEMENOR A REMOVEMENOR A FIM

OBS: this tests if REMOVEMENOR operation is working correctly. The first tests if using REMOVEMENOR in a polynomial with no terms causes any problems, the second one test it when the polynomial has only one term, and the third tests when the polynomial has 2 or more terms.

DEF A 2 4 6 1 8 REMOVE A 7 FIM

DEF A 0 REMOVE A 0 FIM

DEF A 2 3 4 8 5 REMOVE A 4 REMOVE A 5 FIM

OBS: this tests if REMOVE operation is working correctly.

DEF A 0 DEF B 0 FIM

DEF A 2 3 4 7 4 DEF B 9 4 5 1 SOMA A B R FIM

DEF A 2 3 4 7 4 DEF B 9 4 5 1 PROD A B R FIM

OBS: this tests if FIM operation is working correctly